

AUTOMATED REVIEW RATING SYSTEM

PROJECT OVERVIEW

The Automated Review System processes and analyzes customer reviews to predict ratings or sentiment automatically. It cleans and preprocesses review data, splits it for training and testing to ensure reliable evaluation, and uses machine learning models to classify reviews. This system provides businesses with fast, data-driven insights into customer satisfaction, product quality, and areas for improvement, enabling better decision-making and enhanced customer experience.

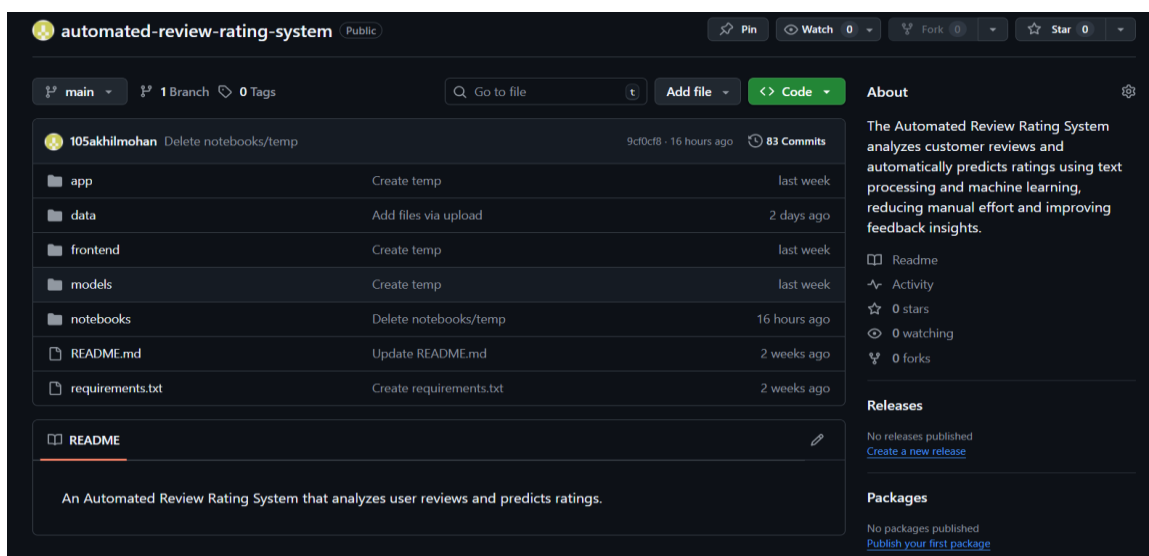
ENVIRONMENTAL SETUP

The project is executed in Google Colab, which provides a cloud-based Python environment with free CPU and GPU resources. The setup includes:

- Python Version: Python 3.x (pre-installed in Colab)
- Data Handling & Preprocessing: pandas, numpy
- Text Processing & NLP: scikit-learn (for train-test split, feature extraction, and machine learning models)
- Visualization: matplotlib, seaborn (for plotting data distributions and results)

GITHUB SETUP

The project was set up on GitHub, a new repository was created to store datasets, preprocessing scripts, and model files. The setup process included initializing the repository, configuring Git on the local system, and pushing the project files to GitHub. A screenshot of the GitHub repository setup is included below to demonstrate the successful configuration and repository structure for the Automated Review Rating System.



DATA COLLECTIONS

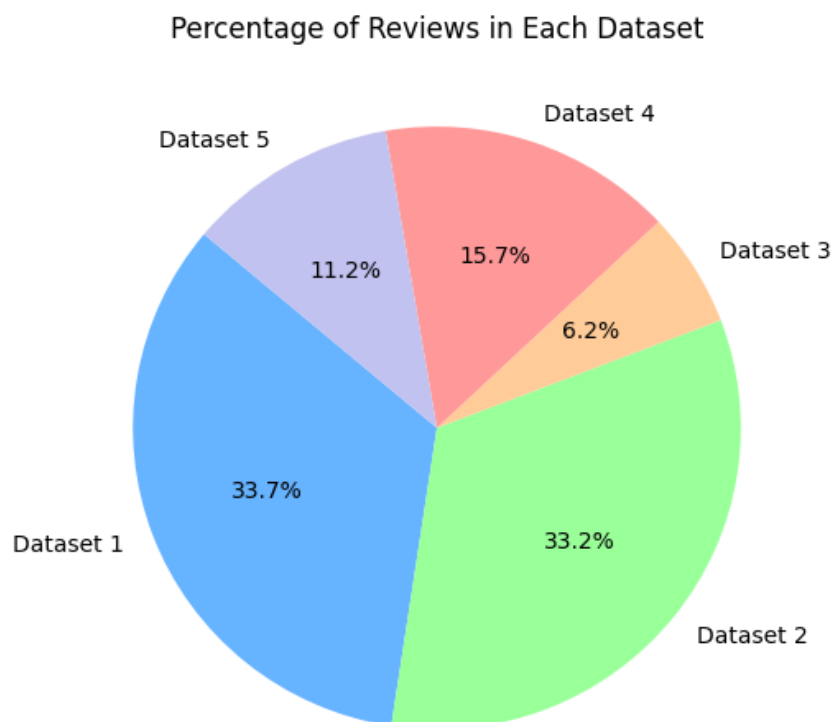
The dataset for this project was sourced from several publicly available Kaggle repositories, encompassing a diverse range of reviews for sentiment analysis:

- Amazon Reviews for Sentiment Analysis: A collection of Amazon product reviews in fastText format, facilitating sentiment classification tasks. [kaggle](#) [kaggle](#)
- ChatGPT Users' Reviews: User feedback on ChatGPT, including textual comments, ratings, and review dates, offering insights into user experiences. [kaggle](#)
- Sentiment Analysis Dataset: A dataset containing labeled sentiment data, suitable for training and evaluating sentiment analysis models. [kaggle](#)
- Reviews Dataset: A compilation of reviews from various industries, providing a broad spectrum of textual data for analysis. [kaggle](#)

The collected datasets, including Amazon product reviews, ChatGPT user reviews, and various sentiment-labeled review datasets, have been uploaded and made publicly available on GitHub for reproducibility and ease of access. [github](#)

Pie-chart

A pie chart was created to visualize the distribution of reviews across five different datasets. The chart shows the relative percentage of reviews contributed by each dataset, highlighting their proportion in the combined corpus



DATA PREPROCESSING

In this stage, several cleaning and transformation steps were performed to ensure data quality and consistency.

Handling Missing Data

Missing values in the dataset were identified and appropriately handled to maintain data consistency and avoid bias during model training.

```
df = df.dropna(subset=["userName", "content", "score"])  
df["thumbsUpCount"] = df["thumbsUpCount"].fillna(0)
```

Removing Duplicates

Duplicate entries were removed to ensure that repeated reviews did not distort the analysis or affect the accuracy of the system.

```
df = df.drop_duplicates()
```

Feature Engineering

New features such as review length and sentiment indicators were created to enhance the dataset's representation and improve model performance.

```
df["review_length"] = df["content"].apply(lambda x: len(str(x).split()))
```

Removing Very Short Reviews

Reviews containing fewer than three words were removed, as they carry little semantic meaning and do not contribute effectively to the analysis.

```
df = df[df['Review'].apply(lambda x: len(str(x).split()) > 4)]
```

Removing Very Long Reviews

Reviews exceeding 100 words were removed from the dataset to ensure consistency and maintain manageable input lengths for analysis. This helps improve processing efficiency and model performance by focusing on concise, meaningful reviews.

```
df = df[df['review'].apply(word_count) <= 100]
```

Drop Blank Reviews

Blank reviews, which contain no meaningful text, were identified and removed from the dataset during preprocessing. This step ensures that only valid and informative reviews are retained for analysis and model training.

```
df = df[df['Review'].notna() & (df['Review'].str.strip() != "")]
```

Removing Special Characters and Emojis

Unnecessary special characters and emojis were eliminated from the text to standardize the review content and make it more suitable for text processing.

```
text = re.sub(r'[^A-Za-z0-9 ]+', "", text)
```

Remove conflicting reviews

Conflicting reviews, where the same user provided multiple reviews with different ratings for the same product, were identified and removed. This step ensures consistency in the dataset and prevents misleading information from affecting the training and evaluation of the model.

```
conflicts = df.groupby('Review')['Rating'].nunique()  
conflict_reviews = conflicts[conflicts > 1].index  
df = df[~df['Review'].isin(conflict_reviews)]
```

Removing URLs

URLs present in reviews were removed since they do not contribute to sentiment or review quality and could introduce noise into the dataset.

```
df['review'] = df['review'].str.replace(r'http[s]?://\S+|www\.\S+', '', regex=True)
```

Remove other language

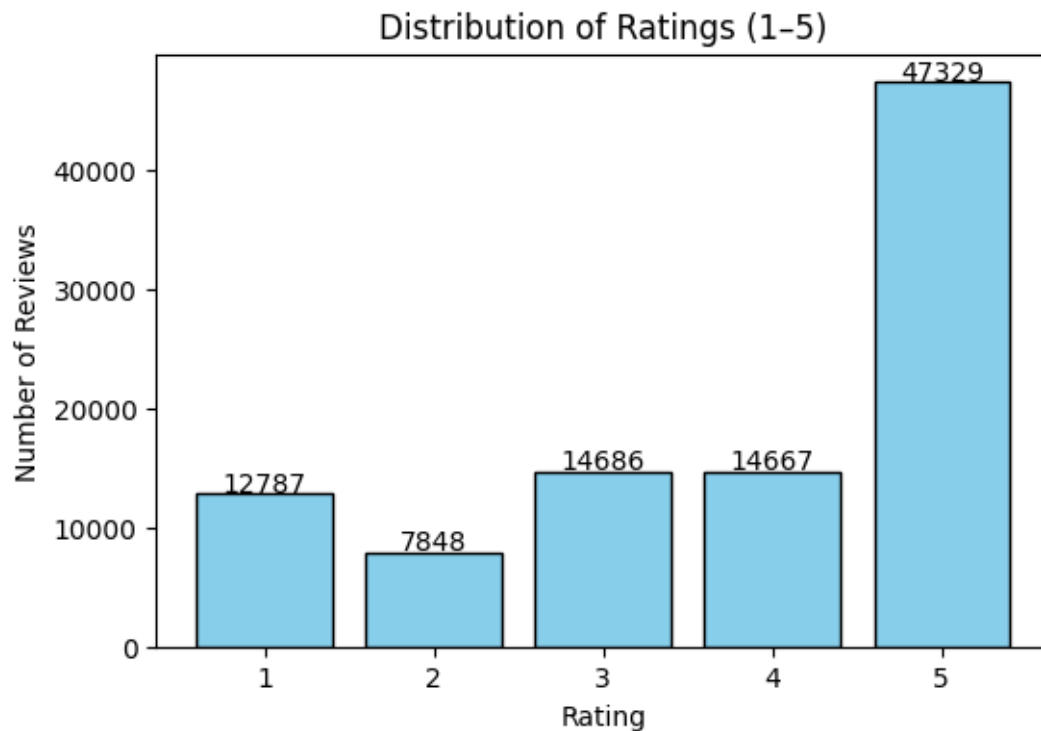
To maintain consistency and improve model performance, reviews in languages other than English are removed. This ensures that the dataset contains only English text for preprocessing and training.

```
english_df = df[df["language"] == "en"].drop(columns=["language"]).reset_index(drop=True)
```

The cleaned and preprocessed dataset, ready for training and analysis, has been uploaded to GitHub for easy access. It includes all reviews with duplicates, very short entries, URLs, special characters, and emojis removed, along with added features like review length. The dataset can be accessed here. [github](#)

DATA VISUALIZATION

After preprocessing, a bar chart was generated to display the distribution of reviews across different rating categories. This visualization helps in understanding the balance of the dataset and ensures that preprocessing steps such as removing duplicates, short reviews, and special characters did not distort the class proportions.



CONVERTING TO LOWERCASE

All review text was converted to lowercase to ensure uniformity and reduce duplication caused by capitalization differences.

```
df = df.applymap(lambda s: s.lower() if isinstance(s, str) else s)
```

BALANCED DATASET

To ensure that the model is trained on an unbiased dataset, a balanced dataset was created by selecting 2000 reviews for each rating from 1 to 5. This resulted in a total of 10,000 reviews, ensuring equal representation across all rating categories.

Code

```
balanced_parts = []  
  
for rating, remaining_df in zip(imbalanced_counts.keys(), remaining_parts):  
    balanced_sample = remaining_df.sample(n=2000, random_state=42)  
    balanced_parts.append(balanced_sample)  
  
balanced_df = pd.concat(balanced_parts).sample(frac=1, random_state=42).reset_index(drop=True)
```

DATA VISUALIZATION

A bar chart was plotted to visualize the distribution of reviews in the balanced dataset. As expected, each rating category has an equal number of reviews, confirming that the dataset is balanced and ready for model training.



SAMPLES OF 1 STAR RATING

Review 1:

new version so bad old account cant login by facebook bad bad

Review 2:

this app isnt letting me use it anymore after i asked it about human nutrition facts and it just said request is not allowed please try again later now it wont even let me tyoe anything new im stuck on a screen that says the same thing

Review 3:

no messages have come through or gone through for over 12hrs im trying to run a business here that requires sending receiving message edit still not working properly having to use messenger lite which has too many limits just get this app working like it was before stop changing stuff when its working perfectly

Review 4:

currently have a bug none of the images gets displayed i also cannot send images no device sync is terrible encryption is only over the wire our history is stored in plain text so its an illusion of security

Review 5:

does not give accurate answers for exams and it is kinda bad at grammar

SAMPLES OF 2 STAR RATING

Review 1:

had a very bad experience talking to your customer executive please make sure that a highly rated platform like freecharge do have some well trained and gentle telecallers

Review 2:

it forces all music to be played with this app i hate it

Review 3:

my zip was approved for 5000 but it is showing as not eligible

Review 4:

its good however the problem i have with this app is whenever i download music it doesnt show in the list of songs and its frustrating that i have to go to directories and play it in the download folder no matter how many times i remove the song and add it back nothing changes its still not showing in the list of songs

Review 5:

audio capture is not functioning

SAMPLES OF 3 STAR RATING

Review 1:

this app has became very horrible since a few days its servers are not working they say that there is internal issue i do also have a stable wifi connection of airtel x stream there wifi service is very good at a pocket freindly price and internet speed is also very good but the airtel app is horrible it have many bugs and glitches

Review 2:

the pop up player does not have a slider to forwardrewind the video

Review 3:

its a good app i entered after a really long time but my friend couldnt it stops at the part of entering her name and profile and says couldnt connect please try again later it happened for me too but suddenly it allowed me after a week of trying can you please fix this as soon as possible

Review 4:

i have paid subscription when i watch movies every 5 mins there are advertisements without sound which i have to forcibly watch what the point of paid subscription

Review 5:

newly added music files arent shown when i select the refresh option they sometimes show up when i set sort by to recently added but this needs to be done at least twice to have any effect also it would be nice to have a feature to edit the song data like title artist etc

SAMPLES OF 4 STAR RATING

Review 1:

chat gpt is a very good app but can not solve mathematics without too much guide lines

Review 2:

forces me to disclose my phone number edit not anymore i signed with my google account at does not ask for a phone number thank you for listening

Review 3:

everything is great except one my issue is that subtitles are not supported while casting please add this feature and this app will be the best

Review 4:

best ai in the world but i wish you can search problems with photos

Review 5:

nice response is good and you really really have to test it but you have to have patience because you know what sometimes those does i mean does mistakes like some mistakes so challenge ability upgrade died and that they i guess you will release 4 point to update i mean this is the version of 35 i know so i guess its cool its its good it can schedule your day you can ask any questions like that so bye

SAMPLES OF 5 STAR RATING

Review 1:

too good the voice feature is brilliant

Review 2:

very valuable for my work and personal use

Review 3:

very good good good good good

Review 4:

its out of this world

Review 5:

its so good and helping my studyi think its effective for each and everyone

STOPWORD REMOVAL

After creating the balanced dataset, stopwords removal was applied to clean the review text. Stopwords are common words (like "the", "is", "and") that do not carry significant meaning for sentiment or rating analysis. Negations such as “no”, “not”, and “never” were retained to preserve sentiment context. This step helps reduce noise in the dataset and improves the effectiveness of subsequent feature extraction and model training.

SpaCy is used in the preprocessing pipeline to efficiently tokenize and lemmatize the text data. By leveraging `nlp.pipe`, the system processes large batches of reviews quickly while removing non-alphabetic characters and stopwords, preserving meaningful words and negations for accurate sentiment analysis.

Code

```
# Load spaCy model

nlp = spacy.load("en_core_web_sm", disable=["parser", "ner", "textcat"])

# Define custom stop words (keep negations)

stop_words = set(stopwords.words('english'))

negations = {"no", "not", "nor", "never", "n't"}

stop_words = stop_words - negations
```

Removed words

During preprocessing, stopwords common words like “the,” “and,” and “is” that carry little semantic meaning were identified in the dataset. After excluding negations such as “not” and “never,” a total of 130 unique stopwords were found across all reviews. Removing these stopwords helps in focusing on meaningful words, improving the efficiency and accuracy of text analysis.

Stopwords

['a', 'about', 'above', 'after', 'again', 'against', 'all', 'am', 'an', 'and', 'any', 'are', 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'd', 'did', 'do', 'does', 'doing', 'don', 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'has', 'have', 'having', 'he', 'her', 'here', 'him', 'himself', 'his', 'how', 'i', 'if', 'in', 'into', 'is', 'it', 'its', 'itself', 'just', 'll', 'm', 'ma', 'me', 'more', 'most', 'my', 'myself', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'our', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'she', 'should', 'so', 'some', 'such', 't', 'than', 'that', 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'we', 'were', 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', 'y', 'you', 'your', 'yours', 'yourself', 'yourselves']

Why is stemming preferred over lemmatization?

Stemming is often preferred over lemmatization when speed is more important than linguistic accuracy. Unlike lemmatization, which converts words to their correct dictionary forms using vocabulary and grammar rules, stemming simply chops off word endings to reduce words to a root form. This makes stemming faster and less resource-intensive, especially for large datasets, though it may produce non-standard or incomplete words. It is useful in applications like search engines or text classification where exact word forms are less critical.

Lemmatization

Lemmatization is a text preprocessing technique used to reduce words to their base or dictionary form, called a lemma. Unlike stemming, which may produce incomplete or non-dictionary words, lemmatization preserves the actual meaning of words. This is particularly useful in sentiment analysis and review processing, as it ensures that different forms of a word are treated consistently. For example, the words “running”, “ran”, and “runs” are all converted to “run”, and “better” is converted to “good”. By applying lemmatization, the model can better understand the semantic content of the reviews.

Code

```
def preprocess_doc(doc):  
    tokens = [  
        token.lemma_ for token in doc  
        if token.is_alpha and token.text not in stop_words  
    ]  
    return " ".join(tokens)
```

SPLITTING THE BALANCED DATASET

After creating a balanced dataset with 2000 reviews for each rating, the dataset was split into training and testing sets to prepare for model development. 80% of the data was allocated for training the model, while 20% was reserved for testing its performance. This ensures that the model is trained on a representative sample while also allowing evaluation on unseen data to assess generalization.

Code

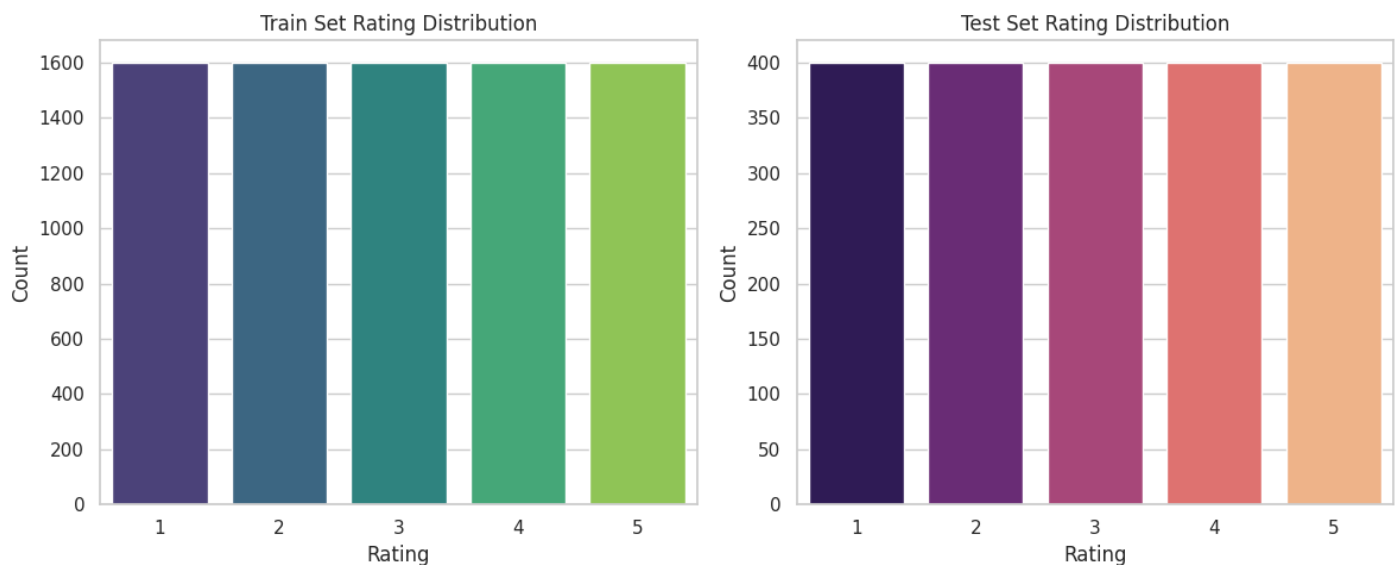
```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y
```

)

The preprocessed train (80%) and test (20%) datasets are available on GitHub for access. [github](#)

DATA VISUALIZATION

To verify the distribution of reviews after splitting, a **bar chart** was plotted showing the count of each rating in the training and testing sets. The chart confirms that both sets maintain a proportional distribution of ratings, thanks to stratified splitting.



TF-IDF CALCULATION

Term Frequency–Inverse Document Frequency (TF-IDF) is a numerical statistic that reflects how important a word is to a document relative to a collection of documents (corpus). It helps highlight significant words while reducing the weight of common words. TF-IDF is calculated as:

$$TF(t,d) = \left(\frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d} \right)$$
$$IDF(t) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t} \right)$$

Code

```
# Initialize TF-IDF

tfidf = TfidfVectorizer(max_features=5000, stop_words='english')

# Fit only on training data
```

```
X_train_tfidf = tfidf.fit_transform(train_df['review'])

# Transform test data using the same vocabulary and IDF

X_test_tfidf = tfidf.transform(test_df['review'])
```

MODEL PERFORMANCE ON TRAINING

The evaluation of multiple machine learning models trained on a balanced text review dataset to predict sentiment or rating categories. The balanced dataset ensures equal representation of all classes, allowing a fair comparison of model performance without bias toward dominant classes. Five models are Logistic Regression, Naive Bayes, Decision Tree, Random Forest, and SVM were assessed using key metrics such as accuracy, precision, recall, and F1-score to determine their effectiveness and generalization capability for text classification tasks.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.4215	0.4111	0.4215	0.4152
Naive Bayes	0.4125	0.4124	0.4125	0.4108
Decision Tree	0.3200	0.3189	0.3200	0.3188
Random Forest	0.4095	0.3871	0.4095	0.3872
SVM	0.4330	0.4224	0.4330	0.4261

Among all models trained on the balanced dataset, SVM achieved the highest overall performance with an accuracy of 0.433, followed closely by Logistic Regression at 0.4215. Both models demonstrated strong precision-recall balance, indicating effective generalization across all rating classes. Tree-based models like Decision Tree and Random Forest showed lower performance, suggesting that linear models are better suited for the TF-IDF-based text representation used in this dataset.

Fine-Tuning Results on Balanced Dataset

The fine-tuning results of various machine learning models trained on the balanced dataset to optimize their performance. Different hyperparameters were tested for each model to identify the most effective configuration. The evaluation focuses on accuracy improvements achieved through tuning parameters such as regularization strength, kernel type, tree depth, and smoothing factors. The results highlight how each model responds to parameter changes and help determine the best-performing model configuration for balanced text classification.

Model	Hyperparameter	Accuracy	Best Parameters
Logistic Regression	C = 0.01 - 0.4085 C = 0.1 - 0.4185 C = 1 - 0.4175 C = 10 - 0.3980	0.4175	C = 1

SVM	kernel = linear - 0.4325 kernel = poly - 0.3780 kernel = rbf - 0.4230 kernel = sigmoid - 0.4275	0.4325	kernel = linear, C = 1
Decision Tree	max_depth = 5 - 0.3090 max_depth = 10 - 0.3320 max_depth = 20 - 0.3375 max_depth = None - 0.3215	0.3375	max_depth = 20, criterion = 'gini'
Random Forest	n_estimators = 50 - 0.4035 n_estimators = 100 - 0.4020 n_estimators = 200 - 0.4020	0.4145	n_estimators = 100, max_depth = 20
Naïve Bayes	alpha = 0.1 - 0.4030 alpha = 0.5 - 0.4075 alpha = 1.0 - 0.4115 alpha = 2.0 - 0.4165	0.4165	alpha = 2.0

The SVM model achieved the highest accuracy after fine-tuning, demonstrating its superior performance on the balanced dataset.

Cross-Test Performance (Balanced - Imbalanced)

When the model trained on the balanced dataset was tested on the imbalanced dataset, it achieved a **cross-test accuracy of 0.4550**, with a **weighted precision of 0.4768** and a **weighted F1 score of 0.4530**. These results indicate that the model generalizes reasonably well to a different data distribution, maintaining strong overall performance despite class imbalance.

Which model would you deploy and why?

Based on the evaluation and fine-tuning results, SVM with a linear kernel (C=1) is the recommended model for deployment. It achieved the highest accuracy (0.4325) on the balanced dataset, maintained strong performance during cross-testing on the imbalanced dataset (accuracy 0.4550), and demonstrated a good balance of precision and F1-score. SVM is well-suited for high-dimensional TF-IDF text features, providing robust generalization across classes. Its linear decision boundary ensures efficient computation and scalability, making it the best choice for reliable, production-ready text classification in this scenario.